Custom Sensitive Information Types — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Custom Sensitive Information Types (Microsoft Purview) 4

What you get . . . 4

Why it matters . . . 4

The bottom line . . . 5

1. Solution Overview — Custom Sensitive Information Types (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

The building blocks of a custom SIT . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Custom Sensitive Information Types (Microsoft Purview) 9

2.1 The org-specific identifier(s) and their exact format, from the data owner . . . 9

2.2 Curated positive AND negative validation samples . . . 9

2.3 Purview roles for creating and managing sensitive information types . . . 9

2.4 Security & Compliance PowerShell (Connect-IPPSSession) and Graph . . . 10

2.5 Decision on the primary element and confidence design . . . 10

2.6 Data owner and stakeholder engagement . . . 10

2.7 A consuming policy and a pilot/simulation scope . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 13

3. Implementation Guide — Custom Sensitive Information Types (Microsoft Purview) 13

Step 1 — Confirm readiness and inventory existing SITs (Assess) . . . 14

Step 2 — Design the SIT (Design) . . . 14

Step 3 — Create the SIT (Configure — build the detector) . . . 14

Step 4 — Validate against positive/negative samples (Validate — the precision gate) . . . 15

Step 5 — Tune for precision (Validate — iterate) . . . 15

Step 6 — Pilot inside a consuming policy in SIMULATION/TEST (Pilot) . . . 16

Step 7 — Enforce and expand (Expand — enforce) . . . 16
5. Testing Guide — Custom Sensitive Information Types (Microsoft Purview) 17
Test 1 — Toolchain and SIT cmdlets work . . . 17
Test 2 — The SIT exists and is correctly composed . . . 17
Test 3 — Positive samples match at the intended confidence (true positives) . . . 17
Test 4 — Negative / look-alike samples do NOT match (false-positive check) . . . 17
Test 5 — Character proximity is doing its job . . . 17
Test 6 — Publishing latency accounted for . . . 18
Test 7 — The consuming policy runs in SIMULATION (no enforcement) . . . 18
Test 8 — Real-content precision in simulation . . . 18
Test 9 — "Where-used" inventory is accurate . . . 18
Test 10 — Safe automation defaults . . . 18
6. Troubleshooting Guide — Custom Sensitive Information Types (Microsoft Purview) 18
The "Create sensitive info type" option isn't visible in the portal . . . 19
`Get/New/Set-DlpSensitiveInformationType*` cmdlets are not recognised . . . 19
`Test-DataClassification` is not available . . . 19
`CLSCUSTOM-Configuration.ps1` didn't import anything . . . 19
Rule-package import fails: schema / validation error . . . 19
The SIT exists but positives don't match (false negatives) . . . 19
Look-alike content matches (false positives) . . . 20
Validation passed but the pilot policy over/under-matches on real content . . . 20
A change to the SIT didn't take effect . . . 20
`CLSCUSTOM-Report.ps1` shows the SIT isn't referenced anywhere . . . 20
`CLSCUSTOM-Report.ps1` shows the SIT in an ENFORCING policy unexpectedly . . . 20
I need to delete a custom SIT / rule package . . . 20
"Does detecting this identifier mean we must retain it / place a hold / report it?" . . . 21
7. Best Practices — Custom Sensitive Information Types (Microsoft Purview) 21
8. Deliverables Summary — Custom Sensitive Information Types (Microsoft Purview) 22
Documentation . . . 22
PowerShell scripts . . . 23
Artifacts this solution creates (naming convention) . . . 23
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Custom Sensitive Information Types (Microsoft Purview)

Microsoft Purview ships hundreds of built-in sensitive information types — but they only recognize data whose format is *standard*. They do **not** recognize the identifiers unique to **your** organization: your employee IDs, customer/account numbers, case and matter numbers, part numbers, contract IDs, and project codenames. Until those patterns are taught to Purview, every DLP, auto-labeling, retention, Insider Risk, and DSPM control that keys off classification is **blind to your most organization-specific data** — a compliance gap and a data-risk exposure hiding in plain sight.

This solution closes that gap. It delivers **custom sensitive information types** end to end — built from **regex**, **keyword lists/dictionaries**, **supporting evidence**, **character-proximity**, and **confidence levels** — then **validates** each one against known **positive and negative** samples, pilots it inside a consuming policy in **simulation/test**, tunes it for precision, and enforces and reuses it across the tenant. It uses only the **Microsoft Purview portal**, **Security & Compliance PowerShell**, and **Microsoft Graph** — no third-party tooling — and is written for a competent Microsoft 365 / compliance administrator working **with** the data owner.

What you get

• A plain-language **Solution Overview** of custom SITs — the building blocks (primary element, supporting evidence, character proximity, confidence levels), how a SIT is *reused* across DLP, labeling, retention, Insider Risk, and DSPM, and the crucial distinction that a **SIT detects, it does not act**.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite: the identifier's exact format, curated **positive and negative** samples, the SIT-management role, the PowerShell/Graph toolchain, the primary-element and confidence design, the data owner, and a consuming policy with a pilot scope.

• An **Implementation Guide** that designs the detector, builds it (portal *or* rule-package XML), **validates it against samples** (the precision gate), pilots it in a **simulation** policy on real content, tunes it, and only then enforces and expands — with pass/fail gates and rollback at every step.

• **Five PowerShell tools** (Security & Compliance PowerShell / Graph): a discovery/inventory script, a rule-package import helper, a `Test-DataClassification`**-based validation** script that scores the SIT against positive/negative samples, a where-used report, and a rule-package export/update helper — **read-only or** `-WhatIf` **by default, no secrets**.

• **Testing** and **Best Practices** that prove precision on positives *and* look-alikes before anything enforces.

Why it matters

• **The gap is real and unmonitored.** The data that is most proprietary — your own identifiers — is exactly the data no built-in SIT can see. Custom SITs are the only way to bring it under Purview.

• **Precision is the value.** A loose SIT floods DLP with false positives and trains users to ignore

warnings; a tight one misses the leak. This solution tunes to **usable precision**, proven on real samples, not guesswork.

- **Build once, reuse everywhere.** A single well-built custom SIT sharpens every downstream classifying

policy — DLP, auto-labeling, retention, Insider Risk, Communication Compliance, DSPM.

- **Pilot-first, always.** A SIT feeds high-impact policies; the consuming policy is validated in

simulation/test on real content before it ever enforces.

The bottom line

Your organization's **unique identifiers detected in Microsoft Purview** — configured, validated against positive and negative samples, piloted, and tuned for precision — a reusable, defensible detector that makes every data-protection control sharper. A fraction of the cost of a consulting engagement, self-guided and tested.

The capabilities that consume custom SITs (advanced DLP, auto-labeling, Insider Risk, DSPM) are licensing-dependent (Microsoft 365 E5 / E5 Compliance; some in preview) and the Purview portal, rule-package schema, and cmdlets evolve; verify capability availability and portal paths against your own licensing and tenant. Whether detected data drives retention, records, or legal actions is deferred to your compliance, records, and legal functions.

1. Solution Overview — Custom Sensitive Information Types (Microsoft Purview)

What this solution is

A **sensitive information type (SIT)** is the pattern-matching classifier Microsoft Purview uses to recognize a category of sensitive data — a credit-card number, a national ID, a health record — inside files, email, chats, and endpoints. Purview ships **hundreds of built-in SITs**, but they only cover data whose format is *standard*. They do **not** recognize the identifiers that are unique to **your** organization: your **employee ID** format, your **customer/account number** scheme, your **project codenames**, your **internal case/matter numbers**, your **part numbers**, or your **contract IDs**. Until those patterns are taught to Purview, every DLP policy, auto-labeling policy, retention rule, Insider Risk indicator, and Copilot/DSPM signal that keys off classification is **blind to your most organization-specific data**.

This solution delivers **custom sensitive information types** end to end: it defines each org-specific identifier as a SIT built from a **primary element** (a **regex** pattern, a **keyword list**, or a **keyword dictionary**), optional **supporting elements** (corroborating keywords/regex found within a **character-proximity** window), and one or more **confidence levels** (low / medium / high) — then **validates** the SIT against known **positive and negative** samples, pilots it by attaching it to a DLP or auto-labeling policy in **simulation/test mode**, tunes it for precision, and only then enforces and expands. It uses only the **Microsoft Purview portal** (`https://purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and — for usage reporting — **Microsoft Graph**. No third-party tooling.

> ***A custom SIT is a *detector*, not an *action*.*** *Creating a SIT does not, by itself, block, label, retain, or alert on anything — it only makes the pattern *available* to reference in a DLP policy, an auto-labeling policy, a retention rule, a Communication Compliance / Insider Risk condition, or DSPM. This solution builds and validates the **detector**; the enforcement action is configured in whichever policy consumes it (and that policy is piloted in **simulation/test** first).*

Why this matters

• **Built-in SITs miss org-specific data.** Your employee IDs, account numbers, and project codenames don't match any standard SIT, so the data that is arguably *most* proprietary is the data Purview can't see — an unmonitored compliance gap and data-risk exposure.

• **Precision is the whole game.** A SIT that's too loose floods every downstream DLP policy with **false positives** (blocking legitimate work, training users to click through warnings); a SIT that's too tight has **false negatives** (the leak it existed to catch walks straight past). The value of this solution is a SIT tuned — with real positive/negative samples — to the **confidence level** that gives usable precision.

• **One detector, many uses.** A single well-built custom SIT is reused across **DLP** (endpoint, email, SharePoint/OneDrive, Teams, cloud), **auto-labeling**, **retention**, **Insider Risk**, **Communication Compliance**, and **DSPM** — build it once, correctly, and every capability that classifies data gets sharper.

• **It must be piloted, never switched on blind.** Because a SIT flows into high-impact policies, the house model applies: validate the detector, run the consuming policy in **simulation/test**, confirm precision on real content, then enforce and expand.

How it works

1. **Discover the identifier and its format.** With the data owner, pin down the exact pattern of the org-specific identifier — its structure (prefix, length, digits/letters, check character, delimiters) and the words that appear *near* it in real documents ("Employee ID:", "Account #", "Matter No.").

1. **Design the SIT.** Choose the **primary element**: a **regex** for structured identifiers, a **keyword list** for a small fixed vocabulary, or a **keyword dictionary** for a large term set. Add **supporting elements** (corroborating keywords or a second regex) and a **character proximity** window so a bare number in the wild doesn't match but "Employee ID: EMP-123456" does. Assign **confidence levels** (High/Medium/Low = 85/75/65) reflecting how much corroborating evidence each pattern requires.

1. **Create the SIT.** In the Purview portal (**Data classification → Classifiers → Sensitive info types → Create**) or by importing a** rule package XML** via Security & Compliance PowerShell (`New-DlpSensitiveInformationTypeRulePackage`). Artifact: `Custom SIT – <Identifier> – Pilot`.

1. **Validate against known samples.** Before trusting it, test the SIT against a curated **positive** set (content that *should* match) and a **negative** set (look-alike content that must *not* match) using `Test-DataClassification` — measuring true/false positives and the confidence returned. This is the precision gate.

1. **Pilot in a consuming policy in SIMULATION/TEST.** Reference the validated SIT from a DLP policy

(or auto-labeling policy) run in **simulation / test / disabled** mode over a limited scope — see what it *would* have caught on real traffic, without blocking anything.

1. **Tune for precision.** Adjust the regex, supporting evidence, proximity, and confidence threshold used in the policy until false positives are acceptable and true positives are still caught. Re-run validation after each change.

1. **Enforce and expand.** With sign-off, move the consuming policy from simulation to **enforce** and widen scope; reuse the same SIT in other policies (retention, auto-label, Insider Risk). Maintain the SIT as the identifier format evolves.

The building blocks of a custom SIT

Element	What it is	When to use
Primary element — Regex	A regular expression matching the identifier's structure	Structured IDs: employee/account/case/part numbers, contract IDs
Primary element — Keyword list	A fixed set of exact terms	Small, stable vocabulary (a handful of codenames)
Primary element — Keyword dictionary	A large managed term set (up to very large)	Big lists (product SKUs, drug names, all project codenames)
Supporting element	Extra keywords/regex that must appear *near* the primary match	Corroboration to cut false positives (e.g. the label "Employee ID" near the number)
Character proximity	The window (default **300** characters) in which supporting evidence must appear	Ties the number to its context so a random number doesn't match
Confidence level	**High (85) / Medium (75) / Low (65)** per pattern	Higher confidence = more corroborating evidence required = fewer false positives
Recommended confidence	The default confidence a consuming policy uses	Set to the level your validation proved precise

What you can control

• The **identifier(s)** turned into custom SITs and each one's **primary element** (regex / keyword list / dictionary).

• The **supporting evidence** and **character-proximity** window that corroborate a match.

• The **confidence levels** defined and the **recommended** level consuming policies default to.

• The **positive/negative validation sample sets** the SIT is proven against.

• Which **DLP / auto-labeling / retention / Insider Risk** policy consumes the SIT — and that it runs

in **simulation/test first**.

- The **pilot scope** validated before any enforcement.

Limitations (be honest with buyers)

- **A SIT detects; it does not act.** Blocking, labeling, retaining, and alerting are configured in the **consuming policy**, not the SIT. This solution builds and validates the detector and pilots it via a consuming policy; the enforcement decision lives in that policy.
- **Pattern matching is probabilistic.** Regex and keyword matching produce false positives and negatives; the confidence level trades one against the other. There is no "perfect" regex — the goal is **usable precision**, proven on samples, not theoretical completeness.
- **Licensing-dependent downstream.** Creating custom SITs is broadly available, but the capabilities that *consume* them (advanced DLP, auto-labeling, Insider Risk, DSPM) are often **E5 / E5 Compliance** and some are in preview. **Verify availability against your own licensing and tenant** — do not assume a consuming capability is present.
- **Rule-package / cmdlet detail evolves.** Portal paths, the rule-package XML schema, and the `DlpSensitiveInformationType*` / `Test-DataClassification` cmdlets are current to Microsoft's public docs but **evolve** — verify against your tenant before relying on an exact path or parameter.
- **Publishing takes time.** A new or changed SIT can take time to become fully available to policies and to appear in classification results; validate patience into the pilot.
- **No secrets in automation.** The provided scripts use interactive modern-auth sign-in and never embed credentials.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and portal. The standard set (running example: an **Employee ID** identifier):

Artifact	Name
Custom SIT (pilot)	`Custom SIT – Employee ID – Pilot`
Custom SIT rule package	`Custom SIT Rule Package – Pilot`
Positive validation sample set	`Custom SIT Test – Positive`
Negative validation sample set	`Custom SIT Test – Negative`
Validation result report	`Custom SIT Validation – Report`
SIT inventory / evidence snapshot	`Custom SIT Inventory – Evidence`
Consuming DLP policy (simulation)	`Custom SIT DLP – Simulation`
Consuming DLP policy (enforced)	`Custom SIT DLP – Enforce`

The standard deployment model

> *Discover the identifier + its format → design the SIT (primary element, supporting evidence, proximity, confidence) → create it → **validate against positive/negative samples** → pilot it in a consuming policy in **simulation/test** → tune for precision → enforce and expand (reuse across policies) → maintain. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)** — with pass/fail gates and rollback, and regulatory/records/legal determinations deferred to the relevant governance functions.*

2. Requirements Guide — Custom Sensitive Information Types (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant licensing, Purview roles, the PowerShell/Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, and a pilot scope with change control) are in `global-requirements-purview.md` — this guide adds the **custom-SIT-specific** detail. **Licensing, portal paths, the rule-package schema, and cmdlets are plan-dependent and the portal evolves — verify each against your own licensing and tenant.**

2.1 The org-specific identifier(s) and their exact format, from the data owner

Description: For each identifier you will turn into a SIT (e.g. **Employee ID**, **Account/Customer number**, **Case/Matter number**, **Part number**, **Contract ID**, **Project codename**), the **exact format**: structure (prefix, fixed length, letters/digits, delimiters, any **check digit/character**), whether variants exist, and — critically — the **words that appear near it** in real documents ("Employee ID:", "Emp #", "Account No.", "Matter"). This is the raw material for the primary regex/ keyword element and the supporting evidence. **Why:** A SIT is only as good as the pattern it encodes; a guessed format produces both false positives and false negatives. The corroborating keywords are what make a bare number match *in context* only. **Verify:** A written spec of each identifier's format and its surrounding keywords, confirmed by the **data owner** (Requirement 2.6). Ideally, real sample documents to derive the pattern from. **Implement:** Interview the data owner / examine real documents; document the regex-able structure and the near-context keyword list before designing the SIT.

2.2 Curated positive AND negative validation samples

Description: Two curated sample sets: a **positive** set — content that **should** match (real or realistic examples of the identifier in context, `Custom SIT Test – Positive`) — and a **negative** set — **look-alike** content that must **not** match (other numbers of similar shape, the identifier without its context, adjacent formats, `Custom SIT Test – Negative`). **Why:** Precision can only be measured against known truth. The negative set is what catches an over-broad regex; without it you cannot prove the SIT won't flood downstream DLP with false positives. **Verify:** Both sets exist and are labeled; each item's *expected* result (match / no-match) is recorded. Real sample data must be handled per your data-handling policy. **Implement:** Assemble both sets with the data owner. Include hard negatives (numbers of the same length, the ID stripped of its keyword) so validation is meaningful.

2.3 Purview roles for creating and managing sensitive information types

Description: The least-privilege role to **create/edit sensitive information types** — membership in a role group holding the relevant permission, typically **Compliance Administrator** or **Compliance Data Administrator** (which include the *Sensitive Information Type* management permissions), or an equivalent custom role group. Global Administrator is **not** required just to build a SIT. **Why:** SIT creation/import is permission-gated; without the role the portal option and the `DlpSensitiveInformationType*` cmdlets are unavailable. **Verify:** Purview portal → `Settings → Roles & scopes → Permissions` shows your account in a role group that can manage sensitive info types; the **Create sensitive info type** option is visible under `Data classification → Classifiers → Sensitive info types`. `CLSCUSTOM-Discovery.ps1` reports whether the `Get-DlpSensitiveInformationType` cmdlets succeed. **Implement:** Assign the least-privilege role group under change control (`global-requirements-purview.md` §2).

2.4 Security & Compliance PowerShell (Connect-IPPSSession) and Graph

Description: A workstation with the **ExchangeOnlineManagement** module (providing `Connect-IPPSSession`, and the `Get/New/Set/Remove-DlpSensitiveInformationType`, `*-DlpSensitiveInformationTypeRulePackage`, and `Test-DataClassification` cmdlets), plus **Microsoft Graph PowerShell** for usage reporting, used with a modern-auth / MFA-capable admin holding the role in 2.3. **Why:** All five provided scripts run through Security & Compliance PowerShell; importing a custom SIT as a **rule package**, and validating it with `Test-DataClassification`, are done through these cmdlets. **Verify:** `Connect-IPPSSession` succeeds; `Get-DlpSensitiveInformationType` returns the built-in SITs; `Test-DataClassification -TestTextItem 'test' -ClassificationNames 'Credit Card Number'` runs without error. `CLSCUSTOM-Discovery.ps1` performs these checks. **Implement:** `Install-Module ExchangeOnlineManagement`; sign in with the role-holding admin; confirm the SIT cmdlets and `Test-DataClassification` are present.

2.5 Decision on the primary element and confidence design

Description: A design decision per SIT: whether the **primary element** is a **regex** (structured IDs), a **keyword list** (small fixed vocabulary), or a **keyword dictionary** (large term set); what **supporting evidence** (keywords/second regex) corroborates it; the **character-proximity** window (default **300**); and the **confidence levels** (High **85** / Medium **75** / Low **65**) and which is **recommended**. Large keyword dictionaries have size/format constraints — **verify the current limits against Microsoft docs and your tenant. Why:** The primary-element choice and confidence design *are* the SIT's precision. Getting the proximity and supporting evidence right is what separates a usable detector from a noise generator. **Verify:** A documented design per SIT (primary element, supporting keywords, proximity, confidence levels, recommended level) approved before build. **Implement:** Draft the design from Requirement 2.1's format spec; pick regex vs keyword list vs dictionary by the identifier's shape; set proximity and confidence conservatively and prove them in validation (Requirement 2.2).

2.6 Data owner and stakeholder engagement

Description: The **data owner** for each identifier (the business function that owns employee IDs, account numbers, matter numbers, etc.) and the compliance/privacy stakeholders (`global-requirements-purview.md` §5) who confirm the identifier is sensitive and how it should be detected. Where a SIT will feed a **records/retention** or **legal-hold** policy, the records/legal owner is engaged for that determination. **Why:** The data owner defines what the identifier is and validates the

samples; whether the detected data drives retention/records/legal actions is **their determination**, not a technical one — the SIT provides the method and evidence. **Verify:** Named data owner and stakeholders on file; sign-off on the identifier format, the samples, and the sensitivity intent. **Implement:** Identify the owner per identifier; capture the format, samples, and sensitivity intent before building. Defer any retention/records/legal *determination* to the relevant governance function.

2.7 A consuming policy and a pilot/simulation scope

Description: Because a SIT only *detects*, plan the **consuming policy** that will exercise it in the pilot — typically a **DLP policy** (or **auto-labeling** policy) referencing the new SIT, run in **simulation / test / disabled** mode over a **limited scope** (`Custom SIT DLP – Simulation`) before any enforcement — plus a change-control path (`global-requirements-purview.md` §6). **Why:** You validate a SIT's real-world precision by seeing what it *would* catch on live content without acting on it. DLP simulation/test mode and auto-label simulation are exactly this; enforcement before a clean pilot is the high-impact mistake the model exists to prevent. **Verify:** A named consuming policy and a limited pilot scope exist; the consuming capability supports **simulation/test** mode (**verify — availability is plan-dependent**); a change window/approval path is defined. **Implement:** Choose the consuming policy and pilot scope; confirm the capability's simulation/test mode is available; agree change control before enabling anything in enforce.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.
- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.
- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Custom Sensitive Information Types (Microsoft Purview)

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com` → **Data classification** → **Classifiers** → **Sensitive info types**) and, where automated, via **Security & Compliance PowerShell** (`Connect-IPPSSession`; the `DlpSensitiveInformationType*` and `Test-DataClassification` cmdlets). Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim. **Portal paths, the rule-package XML schema, and cmdlets are plan-dependent and evolve — verify against your tenant.**

> ***Model:*** *Design the detector, build it,* ***validate it against known positive/negative samples****, pilot it inside a consuming policy in* ***simulation/test****, tune for precision, then enforce and expand. Pattern:* ***Assess → Design → Configure → Pilot (simulate/test) → Validate → Expand (enforce)*** *— with pass/fail gates and rollback.* ***A SIT only detects — every enforcement action lives in the consuming policy, and that policy is piloted in simulation first.*** *Regulatory/records/legal determinations are deferred to the relevant governance functions.*

Step 1 — Confirm readiness and inventory existing SITs (Assess)

1. Confirm the role (Requirement 2.3) and that the **Create sensitive info type** option is visible in

the portal. **Verify entitlement for any downstream consuming capability with Microsoft licensing — do not assume.**

1. Run `CLSCUSTOM-Discovery.ps1` (read-only). It confirms `Connect-IPPSSession` + the SIT cmdlets +

`Test-DataClassification` work, inventories existing **custom** SITs and rule packages (vs built-ins), and writes the `Custom SIT Inventory – Evidence` snapshot — so you don't duplicate an existing SIT.

1. **Pass/fail gate:** cmdlets succeed, the role is present, and you have the current SIT inventory and

the identifier format spec + positive/negative samples (Requirements 2.1, 2.2). Fail → resolve before building.

Step 2 — Design the SIT (Design)

1. With the **data owner** (Requirement 2.6), finalize the design for the running example

`Custom SIT – Employee ID – Pilot`:

• **Primary element:** a **regex** for the identifier's structure — e.g. an Employee ID of the form

`EMP-######` → `\bEMP-\d{6}\b`. (Use a **keyword list** instead for a small fixed vocabulary, or a **keyword dictionary** for a large term set — Requirement 2.5.)

• **Supporting elements:** corroborating keywords that appear near the ID in real documents — e.g.

`Employee ID`, `Emp ID`, `Emp #`, `Personnel No`.

• **Character proximity:** the window supporting evidence must fall within (default **300**).

• **Confidence levels:** define **High (85)** = primary regex **plus** a supporting keyword within

proximity; **Medium (75)** = the regex plus weaker/looser evidence; optionally **Low (65)** = the regex alone. Set the **recommended** confidence to whatever validation proves precise (usually High).

1. Record the design; it drives both the portal build and the rule-package XML.

Step 3 — Create the SIT (Configure — build the detector)

Option A — Portal (recommended for a first SIT):

1. Portal → `Data classification` → `Classifiers` → `Sensitive info types` → `Create sensitive info type`.

2. **Name/description:** `Custom SIT – Employee ID – Pilot`, with a clear description.

3. **Add a pattern** at **High** confidence: set the **primary element** (the regex from Step 2), add a

supporting element (the keyword list), and set the **character proximity**. Add a second pattern at **Medium** confidence with looser evidence if your design calls for it.

1. Set the **recommended confidence level**, then **Test** (Step 4), review, and **Create**.

Option B — PowerShell (rule-package XML):

1. Author the SIT as a **rule package XML** (Entity with `<Pattern confidenceLevel="85">`, an `<IdMatch idRef="Regex_EmployeeID"/>`, a supporting `<Match idRef="Keyword_EmployeeID"/>`, the `<Regex>` and `<Keyword>` definitions, and a `patternsProximity`). Keep the XML in source control.

1. Import it with `CLSCUSTOM-Configuration.ps1` — it **validates the XML**, prints the planned `New-DlpSensitiveInformationTypeRulePackage` (or `Set-…` to update `Custom SIT Rule Package – Pilot`) and runs `-WhatIf` **by default**. Add `-Commit` to import.

1. **Publishing latency:** a new/updated SIT can take time to become fully available to policies and classification — allow for it before concluding a pattern "doesn't work".

1. **Pass/fail gate:** the SIT `Custom SIT – Employee ID – Pilot` exists (portal list or `Get-DlpSensitiveInformationType`) with the intended patterns/confidence. **Rollback:** delete the SIT / remove the rule package (`Remove-DlpSensitiveInformationTypeRulePackage`) — nothing consumes it yet, so blast radius is zero.

Step 4 — Validate against positive/negative samples (Validate — the precision gate)

1. Run `CLSCUSTOM-Validation.ps1` (read-only). It runs `Test-DataClassification` for the SIT against every item in `Custom SIT Test – Positive` and `Custom SIT Test – Negative`, comparing the returned match/confidence to each item's **expected** result, and writes `Custom SIT Validation – Report` (true positives, false positives, false negatives, and the confidence each item returned).

1. Read the report:

- **False positives** (negatives that matched) → the regex/evidence is too broad → tighten (Step 5).
- **False negatives** (positives that didn't match) → the regex/proximity/evidence is too tight → loosen.

1. **Pass/fail gate:** positives match at the intended confidence and negatives do **not** match, at a rate the data owner accepts. Fail → tune (Step 5) and re-validate. **Do not proceed to a consuming policy until this gate passes** — a noisy SIT poisons every policy that later uses it.

Step 5 — Tune for precision (Validate — iterate)

1. Adjust the **regex** (anchors, character classes, length), the **supporting keywords**, the **character proximity**, or the **confidence** design. In PowerShell, edit the rule-package XML and re-import with `CLSCUSTOM-Configuration.ps1` (`-WhatIf` then `-Commit`); use `CLSCUSTOM-Set-Scope.ps1 -Export` first to **back up** the current rule package XML before changing it.

1. Re-run `CLSCUSTOM-Validation.ps1` after **every** change.

2. **Pass/fail gate:** false positives and negatives are both at a level the data owner accepts. Iterate until Step 4's gate passes.

Step 6 — Pilot inside a consuming policy in SIMULATION/TEST (Pilot)

1. Reference the validated SIT from a **consuming policy** run **without enforcement**:

• **DLP:** create a DLP policy `Custom SIT DLP – Simulation` whose rule condition is *content contains*

`Custom SIT – Employee ID – Pilot` at the **recommended** confidence, over a **limited scope**, in **test/simulation (policy tips off / no actions)** mode. **Verify simulation/test mode availability against your licensing.**

• **or Auto-labeling:** create an auto-labeling policy in **simulation mode** using the SIT.

1. Let it run over real content and review what it **would** have matched — without blocking, labeling,

or alerting anyone. Use `CLSCUSTOM-Report.ps1` to list which policies now reference the SIT and the simulation results signals available.

1. **Pass/fail gate:** on real traffic the SIT's matches are overwhelmingly true positives and it isn't

flagging look-alikes. Fail → return to Step 5 (tune the SIT) and/or adjust the policy's confidence threshold. **Rollback:** the consuming policy stays in simulation — nothing is enforced; delete or leave it disabled at any time.

Step 7 — Enforce and expand (Expand — enforce)

1. With data-owner and compliance sign-off, promote the consuming policy from simulation to **enforce**

(`Custom SIT DLP – Enforce`) and **widen scope deliberately** (in rings if large). Use the portal or your DLP tooling; the SIT itself is unchanged.

1. **Reuse the SIT** across other capabilities that need it — additional DLP locations (endpoint, email,

SharePoint/OneDrive, Teams, cloud), **auto-labeling**, **retention** (records/legal *determinations* deferred to those functions), **Insider Risk**, **Communication Compliance**, **DSPM** — each new consuming policy piloted in **simulation/test first**.

1. Stand up the **maintenance cadence**: review the identifier format as it evolves (new prefixes,

length changes), re-validate against refreshed samples, and re-run `CLSCUSTOM-Report.ps1` to keep the inventory and "where-used" evidence current.

1. **Pass/fail gate:** the consuming policy is enforcing on validated true positives, the SIT is reused

correctly, and a re-validation/maintenance cadence is scheduled. **Rollback:** revert the consuming policy to simulation or disable it; the SIT can stay in place for other (still-simulated) uses.

What this guide does NOT decide

• **Whether the detected data triggers retention, records disposition, or legal hold**, and any

regulatory applicability — an **records/compliance/legal determination**. This guide builds and validates the **detector** and pilots a consuming policy; which governance actions the classification drives is decided by those functions.

• **What a DLP/auto-label/Insider-Risk policy *does* on a match** (block, label, retain, alert) — that

is configured and owned in the consuming policy, not the SIT.

5. Testing Guide — Custom Sensitive Information Types (Microsoft Purview)

Run these after building the SIT (Step 3) and again after each tuning change and before enforcing. Every test here is **safe and read-only or contained to a simulation policy** — the point is to prove the **detector's precision** (positives match, look-alikes don't) and that the consuming policy behaves in **simulation** before any enforcement. Handle real sample data per your data-handling policy. **Downstream consuming capabilities are licensing-dependent — verify availability against your tenant.**

Test 1 — Toolchain and SIT cmdlets work

Do: Run `CLSCUSTOM-Discovery.ps1`. **Expected:** `Connect-IPPSSession` succeeds; `Get-DlpSensitiveInformationType` returns SITs; `Test-DataClassification` runs. The `Custom SIT Inventory – Evidence` snapshot lists existing custom SITs and rule packages. **Fail** → install/update ExchangeOnlineManagement, confirm the role (Req 2.3).

Test 2 — The SIT exists and is correctly composed

Do: Run `CLSCUSTOM-Report.ps1` (or `Get-DlpSensitiveInformationType -Identity 'Custom SIT – Employee ID – Pilot'`); review its patterns. **Expected:** The SIT exists with the intended **primary element** (regex/keyword), **supporting evidence**, **character proximity**, and **confidence levels**, and the expected **recommended** confidence. **Fail** → correct via `CLSCUSTOM-Configuration.ps1` (rule-package re-import).

Test 3 — Positive samples match at the intended confidence (true positives)

Do: Run `CLSCUSTOM-Validation.ps1`; review the `Custom SIT Test – Positive` results in `Custom SIT Validation – Report`. **Expected:** Every positive sample **matches** the SIT, and does so at the **expected confidence** level (e.g. High where a supporting keyword is present in proximity). **Fail (false negatives)** → the regex/proximity/evidence is too tight; loosen and re-validate (Implementation Step 5).

Test 4 — Negative / look-alike samples do NOT match (false-positive check)

Do: In the same `CLSCUSTOM-Validation.ps1` run, review the `Custom SIT Test – Negative` results. **Expected:** No negative sample matches — including hard negatives (numbers of the same length, the identifier **without** its surrounding keyword, adjacent formats). **Fail (false positives)** → the regex/evidence is too broad; tighten (anchors, character classes, require supporting evidence, raise the recommended confidence) and re-validate. This is the gate that protects every downstream policy.

Test 5 — Character proximity is doing its job

Do: Validate two crafted items: (a) the identifier **with** its keyword within the proximity window; (b) the identifier **far from** any keyword (outside the window). **Expected:** (a) matches at the higher confidence; (b) matches only at a lower confidence or not at all, per your design. **Fail** → adjust the proximity window /

confidence design (Implementation Step 2/5).

Test 6 — Publishing latency accounted for

Do: If a just-created/changed SIT doesn't yet behave as validated, wait and re-run `CLSCUSTOM-Validation.ps1`. **Expected:** After the SIT has fully published, validation results are stable and repeatable. **Fail after ample time** → re-check the rule-package import and pattern definitions, not just latency.

Test 7 — The consuming policy runs in SIMULATION (no enforcement)

Do: With the SIT referenced by `Custom SIT DLP – Simulation`, run `CLSCUSTOM-Report.ps1` and check the policy state. **Expected:** The consuming policy is in **test/simulation** (or disabled) mode — it reports what it *would* match but **blocks/labels/alerts nothing**. The report shows the SIT is referenced. **Fail** → set the policy to simulation before it processes real content.

Test 8 — Real-content precision in simulation

Do: Let `Custom SIT DLP – Simulation` observe real traffic in its limited scope; review what it *would* have matched (simulation results / policy-tip-only signals). **Expected:** The would-be matches are overwhelmingly **true** instances of the identifier; look-alikes are not flagged. **Fail** → tune the SIT (Step 5) and/or the policy's confidence threshold; re-simulate.

Test 9 — "Where-used" inventory is accurate

Do: Run `CLSCUSTOM-Report.ps1`. **Expected:** The report lists the SIT's patterns/confidence and **every policy that references it** (DLP / auto-label / etc.) — the reuse evidence. **Fail** → confirm the SIT name and that the reporting scope covers the relevant policy types.

Test 10 — Safe automation defaults

Do: Run `CLSCUSTOM-Configuration.ps1` and `CLSCUSTOM-Set-Scope.ps1` with no action switch. **Expected:** Each runs in `-WhatIf` mode and only *describes* the change; nothing is modified until `-Commit` is supplied. `CLSCUSTOM-Discovery.ps1`, `CLSCUSTOM-Validation.ps1`, and `CLSCUSTOM-Report.ps1` are read-only. **Fail** → stop; do not run with `-Commit` until the dry-run output and validation are correct.

Gate to enforce: Tests 1–2 (build), 3–6 (detector precision), 7–8 (simulation on real content), and 9–10 (evidence + safety) all pass → promote the consuming policy from simulation to enforce (Implementation Step 7). Any fail → tune and re-test. Enforcement is a data-owner/compliance-approved decision; retention/records/legal determinations are deferred to those functions.

6. Troubleshooting Guide — Custom Sensitive Information Types (Microsoft Purview)

Field issues and how to resolve them. **Symptom** → **Likely cause** → **Resolution.** Confirm the basics first: the right compliance role (can manage sensitive info types), a successful `Connect-IPPSSession`, and that any *consuming* capability you're piloting is licensed. Portal paths, the rule-package schema, and cmdlets are plan-dependent and evolve — **verify against your tenant.**

The "Create sensitive info type" option isn't visible in the portal

Likely cause: Your account isn't in a role group that can manage sensitive information types. **Resolution:** Add the account to **Compliance Administrator** or **Compliance Data Administrator** (or an equivalent custom role group) — Requirement 2.3. Sign out/in after a role change. Global Admin is not required just to build a SIT.

`Get/New/Set-DlpSensitiveInformationType*` cmdlets are not recognised

Likely cause: Not connected via `Connect-IPPSSession`, or an outdated ExchangeOnlineManagement module. **Resolution:** `Import-Module ExchangeOnlineManagement`; `Update-Module ExchangeOnlineManagement`; `Connect-IPPSSession` (Security & Compliance PowerShell — **not** `Connect-ExchangeOnline`). `CLSCUSTOM-Discovery.ps1` performs this check.

`Test-DataClassification` is not available

Likely cause: Outdated module, or the cmdlet isn't enabled for your tenant/role. **Resolution:** Update ExchangeOnlineManagement and reconnect; confirm your role. If it remains unavailable, validate the SIT in the portal's **Test** experience instead (upload/paste a sample) — the `CLSCUSTOM-Validation.ps1` automation depends on this cmdlet. **Verify availability against your tenant.**

`CLSCUSTOM-Configuration.ps1` didn't import anything

Likely cause: It ran in the default `-WhatIf` mode (by design). **Resolution:** Review the validated XML and the planned action it printed; re-run with `-Commit` *after* `CLSCUSTOM-Validation.ps1` passes. Back up the current package first with `CLSCUSTOM-Set-Scope.ps1 -Export`.

Rule-package import fails: schema / validation error

Likely cause: The XML doesn't match the current rule-package schema (missing `<IdMatch>`, malformed `<Regex>`/`<Keyword>`, bad `confidenceLevel`, duplicate entity **GUID**, or an unescaped character in the regex). **Resolution:** Validate the XML against Microsoft's current *"create a custom sensitive information type in PowerShell"* schema. Ensure every `<Entity>` has a **unique GUID**, each `<Pattern>` has a valid `confidenceLevel` (65/75/85), and regex special characters are properly escaped. `CLSCUSTOM-Configuration.ps1` checks *well-formedness* but not full schema conformance — fix the reported service error and re-import.

The SIT exists but positives don't match (false negatives)

Likely cause: The regex is too strict; supporting evidence is required but absent in the sample; the proximity window is too small; or the SIT hasn't finished **publishing** yet. **Resolution:** Allow publish time and re-run `CLSCUSTOM-Validation.ps1`. Then loosen: relax the regex (anchors, character classes,

length), widen **character proximity**, or make supporting evidence optional at a lower confidence. Re-validate after each change (Implementation Step 5).

Look-alike content matches (false positives)

Likely cause: The regex is too broad, no supporting evidence is required, or the recommended confidence is too low — so any number of that shape matches. **Resolution:** Tighten the regex, **require a supporting keyword within proximity** for the higher confidence level, and set the **recommended confidence** to what validation proves. Add the offending look-alikes to `Custom SIT Test – Negative` and re-run `CLSCUSTOM-Validation.ps1` until FP = 0. This is the gate that protects every downstream policy.

Validation passed but the pilot policy over/under-matches on real content

Likely cause: Real documents differ from the samples; the consuming policy uses a different confidence threshold than validated; or scope/coverage differs. **Resolution:** Confirm the consuming policy references the SIT at the **recommended** confidence (`CLSCUSTOM-Report.ps1`). Refresh the sample sets from real content, re-validate, and adjust the policy's confidence threshold. Keep the policy in **simulation** until precision on real traffic is acceptable.

A change to the SIT didn't take effect

Likely cause: Publishing latency, or the update was applied to a different rule package / name. **Resolution:** Allow time; re-run `CLSCUSTOM-Validation.ps1`. Confirm you updated the correct package (`CLSCUSTOM-Discovery.ps1` lists them) and used `-Commit`. A dry-run (`-WhatIf`) changes nothing.

`CLSCUSTOM-Report.ps1` shows the SIT isn't referenced anywhere

Likely cause: The consuming policy references a **different SIT name**, or it's an auto-labeling / non-DLP consumer this report doesn't enumerate. **Resolution:** Confirm the exact SIT name matches (`CLSCUSTOM-Discovery.ps1`). This report covers **DLP compliance** rules; check **auto-labeling** and other consumers in the Purview portal, which use separate cmdlets.

`CLSCUSTOM-Report.ps1` shows the SIT in an ENFORCING policy unexpectedly

Likely cause: A consuming DLP policy is in **Enable** (enforce) mode rather than **Test** mode. **Resolution:** If the SIT hasn't cleared the validation + simulation gates, set the consuming policy back to **TestWithNotifications / TestWithoutNotifications** (simulation) until it has. Enforcement is the final, approved step — not the default.

I need to delete a custom SIT / rule package

Likely cause: A retired or mistaken SIT. **Resolution:** First confirm nothing consumes it (`CLSCUSTOM-Report.ps1` — and check auto-labeling in the portal); a SIT referenced by a live policy should not be deleted out from under it. Then remove the rule package (`Remove-DlpSensitiveInformationTypeRulePackage`) or delete the SIT in the portal. Back up the XML first (`CLSCUSTOM-Set-Scope.ps1 -Export`).

"Does detecting this identifier mean we must retain it / place a hold / report it?"

Not a tool question. The SIT provides the **detection method and evidence**. Whether the detected data triggers **retention, records disposition, legal hold, or regulatory reporting** is a **records/compliance/legal determination** — defer it to those functions (Requirement 2.6, `global-requirements-purview.md` §5).

7. Best Practices — Custom Sensitive Information Types (Microsoft Purview)

- **Design the detector before you build it.** Pin down the identifier's exact format *and* the words

that appear near it in real documents first. A SIT authored from a guessed pattern generates both false positives and false negatives; one authored from real samples doesn't.

- **Precision beats coverage.** A SIT that's too loose floods every downstream DLP policy with false

positives and trains users to click through warnings — which is worse than not detecting at all. Tune for the confidence level that gives **usable precision** on real content, not maximal theoretical matching.

- **Always validate against negatives, not just positives.** Positives-only testing tells you the SIT

can match; only a curated **negative / look-alike** set proves it won't over-match. Include hard negatives: numbers of the same length, the identifier stripped of its keyword, adjacent formats.

- **Use supporting evidence and proximity.** A bare regex on a number will match everything of that

shape. Requiring a **supporting keyword within the character-proximity window** is what turns "any 6-digit number" into "an Employee ID" — it's the single biggest precision lever.

- **Design confidence levels deliberately.** High (85) should require the primary match **plus**

corroboration; reserve Low (65) for cases where the pattern alone is genuinely unambiguous. Set the **recommended** confidence to the level your validation proved, and have consuming policies use it.

- **A SIT detects — it never acts.** Keep the mental model clean: the SIT is the *detector*; blocking,

labeling, retention, and alerting live in the **consuming policy**. Build/validate the detector here; configure the action there, and pilot that policy in **simulation/test first**.

- **Always pilot the consuming policy in simulation.** Reference the validated SIT from a DLP or

auto-labeling policy run in **simulation/test** over a limited scope, watch what it *would* catch on real traffic, and only then enforce. Never point an enforcing policy at a brand-new SIT blind.

- **Always dry-run automation.** `CLSCUSTOM-Configuration.ps1` and `CLSCUSTOM-Set-Scope.ps1` default to

`-WhatIf`. Read every planned line; run `-Commit` only after the validation gate passes.

- **Version and back up the rule package.** Keep the rule-package XML in source control and

`CLSCUSTOM-Set-Scope.ps1 -Export` a backup before every change — so a bad edit is one restore away and you have a defensible history of the pattern.

• **Build once, reuse everywhere.** A well-tuned custom SIT is reused across DLP (endpoint, email, SharePoint/OneDrive, Teams, cloud), auto-labeling, retention, Insider Risk, Communication Compliance, and DSPM. Invest in one correct detector rather than many rushed ones.

• **Allow for publishing latency.** A new or changed SIT takes time to fully propagate to policies and classification. Don't declare a pattern broken on the first run — validate patience into the pilot.

• **Keep the identifier format current.** Formats drift (new prefixes, longer numbers, new variants). Re-validate against refreshed samples on a cadence and update the SIT before the drift creates blind spots.

• **Determinations belong to the data owner and governance.** Whether the detected data is sensitive, and whether classification drives **retention/records/legal** actions, is the data owner's and governance functions' call — the SIT supplies the method and evidence, not the verdict.

• **Verify licensing and availability against your tenant.** Creating SITs is broadly available, but the capabilities that consume them (advanced DLP, auto-labeling, Insider Risk, DSPM), the rule-package schema, and portal paths are plan-dependent and evolve — confirm, never assume.

8. Deliverables Summary — Custom Sensitive Information Types (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case, the detector-not-action model, and what's in the box
1. Solution Overview	What a custom SIT is, the building blocks (primary element, supporting evidence, proximity, confidence), how it works, reuse across policies, limitations, and the naming convention
2. Requirements Guide	Every custom-SIT-specific prerequisite with Verify / Implement-if-missing (identifier format, positive/negative samples, SIT-management role, toolchain, primary-element/confidence design, data owner, consuming policy + pilot)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + Security & Compliance PowerShell steps — design, build (portal or rule-package XML), validate against samples, pilot in a simulation policy, tune, enforce and reuse; determinations deferred

Document	Purpose
5. Testing Guide	Validation procedures with pass/fail gates (build, detector precision on positives/negatives, proximity, simulation on real content, evidence, safe defaults)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven, precision-first recommendations

PowerShell scripts

Script	Purpose	Default safety
`CLSCUSTOM-Discovery.ps1`	Readiness + SIT-cmdlet check; inventories existing **custom** SITs and rule packages (vs built-ins) and writes the `Custom SIT Inventory – Evidence` snapshot	**Read-only**
`CLSCUSTOM-Configuration.ps1`	Imports/updates a custom SIT from a **rule-package XML** (validates the XML, composes `New/Set-DlpSensitiveInformationTypeRulePackage`)	`-WhatIf` (needs `-Commit`)
`CLSCUSTOM-Validation.ps1`	Runs `Test-DataClassification` for the SIT against the positive/negative sample sets and writes `Custom SIT Validation – Report` (true/false positives, false negatives, confidence)	**Read-only**
`CLSCUSTOM-Report.ps1`	Inventory + **where-used** report — SIT patterns/confidence and every DLP/auto-label policy that references it	**Read-only**
`CLSCUSTOM-Set-Scope.ps1`	Rule-package helper — **export/back up** the current rule-package XML and apply an updated one	`-WhatIf` (needs `-Commit`)

All scripts run through **Security & Compliance PowerShell** (`Connect-IPPSSession`) and/or **Microsoft Graph** with interactive modern-auth; **no credentials or secrets are embedded**. The two configuration scripts are **safe-by-default** (`-WhatIf`) and only change the tenant when `-Commit` is supplied — after the validation gate passes.

Artifacts this solution creates (naming convention)

Artifact	Name
Custom SIT (pilot)	`Custom SIT – Employee ID – Pilot`
Custom SIT rule package	`Custom SIT Rule Package – Pilot`
Positive validation sample set	`Custom SIT Test – Positive`
Negative validation sample set	`Custom SIT Test – Negative`
Validation result report	`Custom SIT Validation – Report`

Artifact	Name
SIT inventory / evidence snapshot	`Custom SIT Inventory – Evidence`
Consuming DLP policy (simulation)	`Custom SIT DLP – Simulation`
Consuming DLP policy (enforced)	`Custom SIT DLP – Enforce`

Outcome

Your organization's **unique identifiers detected in Microsoft Purview** — custom SITs built from regex, keywords, and confidence design, **validated against positive and negative samples**, piloted in simulation, and tuned for precision — a reusable detector that makes every downstream DLP, labeling, retention, Insider Risk, and DSPM policy sharper, with the enforcement action and any governance determination owned by the consuming policy and the relevant functions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLSCUSTOM-Configuration.ps1

Purpose: Imports a new custom sensitive information type (SIT) rule package, or updates an existing one, from a rule-package XML file you author (Entity + Pattern confidenceLevel + IdMatch regex + supporting Match keyword + patternsProximity). Safe by default: with no switch it VALIDATES the XML and prints the planned New-/Set-DlpSensitiveInformationTypeRulePackage without changing anything (-WhatIf). Supply -Commit to apply — only after the validation gate (CLSCUSTOM- Validation.ps1) passes. A SIT only DETECTS; the enforcement action lives in a consuming policy. HOW TO RUN Dry-run (default) — validates the XML and shows the planned import, changes nothing: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Configuration.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml Update an already-imported package (same file, edited): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Configuration.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml -Update Apply (after validation passes): ... same parameters ... -Commit Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Back up the current package first with CLSCUSTOM-Set-Scope.ps1 -Export. EXPECTED RESULTS Dry-run: "XML valid" plus the exact New-DlpSensitiveInformationTypeRulePackage (or Set-... with -Update) it would run. With -Commit, the rule package is imported/updated and echoed back. Verify with CLSCUSTOM-Report.ps1 and validate with CLSCUSTOM-Validation.ps1. .NOTES Purview — Custom Sensitive Information Types. -WhatIf by default; -Commit required to change the tenant. No credentials stored. The rule-package XML schema evolves — verify against Microsoft's current "create a custom sensitive information type in PowerShell" documentation for your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RulePackagePath, # path to the rule-package XML you authored [switch]$Update, # update an existing package (Set-...) instead of creating [switch]$Commit)

How to run: Dry-run (default) — validates the XML and shows the planned import, changes nothing: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Configuration.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml Update an already-imported package (same file, edited): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Configuration.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml -Update Apply (after validation passes): ... same parameters ... -Commit Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Back up the current package first with CLSCUSTOM-Set-Scope.ps1 -Export. EXPECTED RESULTS Dry-run: "XML valid" plus the exact New-DlpSensitiveInformationTypeRulePackage (or Set-... with -Update) it would run. With -Commit, the rule package is imported/updated and echoed back. Verify with CLSCUSTOM-Report.ps1 and validate with CLSCUSTOM-Validation.ps1. .NOTES Purview — Custom Sensitive Information Types. -WhatIf by default; -Commit required to change the tenant. No credentials stored. The rule-package XML schema evolves — verify against Microsoft's current "create a custom sensitive information type in PowerShell" documentation for your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RulePackagePath, # path to the rule-package XML you authored [switch]$Update, # update an existing package (Set-...) instead of creating [switch]$Commit)

CLSCUSTOM-Discovery.ps1

Purpose: Assesses whether the tenant is ready to build custom sensitive information types (SITs) and captures the inventory baseline. Confirms Connect-IPPSSession, the DlpSensitiveInformationType

cmdlets, and Test-DataClassification are available; inventories existing CUSTOM SITs and rule packages (separated from the built-ins) so you do not duplicate one; and writes the "Custom SIT Inventory - Evidence" snapshot. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Discovery.ps1 -CsvPath C:\Temp\CustomSIT-Inventory.csv Requires the ExchangeOnlineManagement module; connects with Connect-IPPSSession (Security & Compliance PowerShell). Sign in with an admin holding a role that can manage sensitive info types (e.g. Compliance Administrator / Compliance Data Administrator). EXPECTED RESULTS A console summary: toolchain OK, the count of built-in vs custom SITs, a list of custom SITs and rule packages (the "Custom SIT Inventory - Evidence" snapshot), and a Test-DataClassification smoke test. Optional CSV of the custom-SIT inventory. .NOTES Purview — Custom Sensitive Information Types. Read-only. No credentials are stored. The capabilities that CONSUME SITs (advanced DLP, auto-labeling, Insider Risk, DSPM) are licensing-dependent and the portal/cmdlets evolve — verify availability against your own licensing and tenant. > [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Discovery.ps1 -CsvPath C:\Temp\CustomSIT-Inventory.csv Requires the ExchangeOnlineManagement module; connects with Connect-IPPSSession (Security & Compliance PowerShell). Sign in with an admin holding a role that can manage sensitive info types (e.g. Compliance Administrator / Compliance Data Administrator). EXPECTED RESULTS A console summary: toolchain OK, the count of built-in vs custom SITs, a list of custom SITs and rule packages (the "Custom SIT Inventory - Evidence" snapshot), and a Test-DataClassification smoke test. Optional CSV of the custom-SIT inventory. .NOTES Purview — Custom Sensitive Information Types. Read-only. No credentials are stored. The capabilities that CONSUME SITs (advanced DLP, auto-labeling, Insider Risk, DSPM) are licensing-dependent and the portal/cmdlets evolve — verify availability against your own licensing and tenant. > [CmdletBinding()] param([string]$CsvPath)

CLSCUSTOM-Report.ps1

Purpose: Reports a custom SIT's definition (patterns, confidence levels, recommended confidence) and, crucially, WHERE IT IS USED — every DLP compliance policy/rule that references it — so you can see the blast radius before enforcing and keep reuse evidence current. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Report.ps1 -SitName 'Custom SIT - Employee ID - Pilot' powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Report.ps1 -SitName 'Custom SIT - Employee ID - Pilot' -CsvPath C:\Temp\CustomSIT-WhereUsed.csv Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Sign in with a compliance role that can read sensitive info types and DLP policies. EXPECTED RESULTS Console: the SIT's definition, then the DLP policies/rules that reference it (name + mode: Enable / TestWithNotifications / TestWithoutNotifications / Disable) — so simulation vs enforce is visible. Optional CSV of the where-used list. .NOTES Purview — Custom Sensitive Information Types. Read-only. No credentials stored. Auto-labeling and other consuming policy types have their own cmdlets; this report covers DLP compliance policies — verify additional consumers in the portal. Portal/cmdlets evolve — verify against your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, # e.g. 'Custom SIT - Employee ID - Pilot' [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Report.ps1 -SitName 'Custom SIT - Employee ID - Pilot' powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Report.ps1 -SitName 'Custom SIT - Employee ID - Pilot' -CsvPath C:\Temp\CustomSIT-WhereUsed.csv Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Sign in with a compliance role that

can read sensitive info types and DLP policies. EXPECTED RESULTS Console: the SIT's definition, then the DLP policies/rules that reference it (name + mode: Enable / TestWithNotifications / TestWithoutNotifications / Disable) — so simulation vs enforce is visible. Optional CSV of the where-used list. .NOTES Purview — Custom Sensitive Information Types. Read-only. No credentials stored. Auto-labeling and other consuming policy types have their own cmdlets; this report covers DLP compliance policies — verify additional consumers in the portal. Portal/cmdlets evolve — verify against your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, # e.g. 'Custom SIT - Employee ID - Pilot' [string]$CsvPath)

CLSCUSTOM-Set-Scope.ps1

Purpose: Rule-package lifecycle helper. -Export writes the CURRENT custom rule package XML to disk (a backup you take BEFORE every change, and your version-control artifact) — read-only. Applying an updated rule package is safe by default: with no switch it shows the planned Set-... and changes nothing (-WhatIf); supply -Commit to apply. Tuning the SIT (regex / supporting evidence / proximity / confidence) is done by editing the exported XML and re-applying, then re-validating. HOW TO RUN Back up the current custom rule package (read-only): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -Export -OutPath .\CustomSIT-RulePackage.backup.xml Dry-run applying an edited package (default) — shows the planned Set-..., changes nothing: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml Apply the edit (after CLSCUSTOM-Validation.ps1 passes): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml -Commit Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. EXPECTED RESULTS -Export: the current custom rule package XML written to -OutPath. Apply dry-run: the planned Set-DlpSensitiveInformationTypeRulePackage. -Commit: the package updated and echoed back. Re-validate with CLSCUSTOM-Validation.ps1 after any change. .NOTES Purview — Custom Sensitive Information Types. -Export is read-only; applying is -WhatIf by default and needs -Commit. No credentials stored. Always back up (-Export) before editing. A SIT only DETECTS — enforcement lives in the consuming policy. Schema/cmdlets evolve — verify against your tenant. > [CmdletBinding(DefaultParameterSetName='Apply')] param([Parameter(ParameterSetName='Export', Mandatory)][switch]$Export, [Parameter(ParameterSetName='Export')][string]$PackageName = 'Custom SIT Rule Package - Pilot', [Parameter(ParameterSetName='Export', Mandatory)][string]$OutPath,

How to run: Back up the current custom rule package (read-only): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -Export -OutPath .\CustomSIT-RulePackage.backup.xml Dry-run applying an edited package (default) — shows the planned Set-..., changes nothing: powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml Apply the edit (after CLSCUSTOM-Validation.ps1 passes): powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Set-Scope.ps1 -RulePackagePath .\CustomSIT-RulePackage.xml -Commit Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. EXPECTED RESULTS -Export: the current custom rule package XML written to -OutPath. Apply dry-run: the planned Set-DlpSensitiveInformationTypeRulePackage. -Commit: the package updated and echoed back. Re-validate with CLSCUSTOM-Validation.ps1 after any change. .NOTES Purview — Custom Sensitive Information Types. -Export is read-only; applying is -WhatIf by default and needs -Commit. No credentials stored. Always back up (-Export) before editing. A SIT only DETECTS — enforcement lives in the consuming policy. Schema/cmdlets evolve — verify against your tenant. > [CmdletBinding(DefaultParameterSetName='Apply')] param([Parameter(ParameterSetName='Export', Mandatory)][switch]$Export,

[Parameter(ParameterSetName='Export')][string]$PackageName = 'Custom SIT Rule Package - Pilot', [Parameter(ParameterSetName='Export', Mandatory)][string]$OutPath,

CLSCUSTOM-Validation.ps1

Purpose: Runs Test-DataClassification for a custom sensitive information type against curated POSITIVE samples (should match) and NEGATIVE / look-alike samples (must not match), compares each result to its expected outcome, and reports true positives, FALSE positives, FALSE negatives, and the confidence each sample returned. This is the precision gate: it must pass before the SIT is referenced by any consuming policy. Read-only — it only TESTS text, it changes nothing. HOW TO RUN Samples CSV with columns: Text,Expected (Expected = Match | NoMatch) powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Validation.ps1 `-SitName 'Custom SIT - Employee ID - Pilot' -SamplesCsv .\CustomSIT-Samples.csv powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Validation.ps1` -SitName 'Custom SIT - Employee ID - Pilot' -SamplesCsv .\CustomSIT-Samples.csv -OutCsv .\Validation-Report.csv Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Handle real sample data per your data-handling policy. EXPECTED RESULTS The "Custom SIT Validation - Report": per-sample match/no-match + confidence, and totals — TP / FP / FN. PASS when positives match (at the intended confidence) and negatives do not. Any FP or FN means tune the SIT (Implementation Step 5) and re-validate. .NOTES Purview — Custom Sensitive Information Types. Read-only (Test-DataClassification only). No credentials stored. Confidence thresholds: High 85 / Medium 75 / Low 65 — verify against your tenant's current behavior. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, # e.g. 'Custom SIT - Employee ID - Pilot' [Parameter(Mandatory)][string]$SamplesCsv, # CSV: Text,Expected (Expected = Match | NoMatch) [int]$MinConfidence = 75, # confidence at/above which a positive counts as matched [string]$OutCsv)

How to run: Samples CSV with columns: Text,Expected (Expected = Match | NoMatch) powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Validation.ps1 `-SitName 'Custom SIT - Employee ID - Pilot' -SamplesCsv .\CustomSIT-Samples.csv powershell -ExecutionPolicy Bypass -File .\CLSCUSTOM-Validation.ps1` -SitName 'Custom SIT - Employee ID - Pilot' -SamplesCsv .\CustomSIT-Samples.csv -OutCsv .\Validation-Report.csv Requires ExchangeOnlineManagement; connects with Connect-IPPSSession. Handle real sample data per your data-handling policy. EXPECTED RESULTS The "Custom SIT Validation - Report": per-sample match/no-match + confidence, and totals — TP / FP / FN. PASS when positives match (at the intended confidence) and negatives do not. Any FP or FN means tune the SIT (Implementation Step 5) and re-validate. .NOTES Purview — Custom Sensitive Information Types. Read-only (Test-DataClassification only). No credentials stored. Confidence thresholds: High 85 / Medium 75 / Low 65 — verify against your tenant's current behavior. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, # e.g. 'Custom SIT - Employee ID - Pilot' [Parameter(Mandatory)][string]$SamplesCsv, # CSV: Text,Expected (Expected = Match | NoMatch) [int]$MinConfidence = 75, # confidence at/above which a positive counts as matched [string]$OutCsv)